

网络与信息安全

about final test:

10个问答题，40个单选题

侧重于对基础概念理解的考查，无复杂计算题和具体操作题，不会涉及过多的技术细节。

40points

1. 网络信息安全概述

一个介绍，重点是在于后面的特性。（老师上课强调）

1.1 网络信息安全事件

1.2 网络信息安全现状

1. 阶段

通信保密阶段、信息安全阶段、网络信息系统安全阶段、？

2. 经典理论：

所有的程序都有缺陷。

大程序的缺陷甚至比包含的内容还多，一个安全想过的程序也会有安全性缺陷

只要不运行这个程序，是否有缺陷，无关紧要

对外暴露的计算机应该尽可能少的运行程序，运行的程序也要尽可能的小。

3. 社会工程学

一种通过对受害者心理弱点、本能反应、好奇心、^{信任}贪婪心等心理陷阱进行诸如诈骗、伤害等危害手段。

社会工程学陷阱就是通常以交谈、欺骗、假冒或者口语等方式，从合法用户中套取用户系统的秘密。

1.3 网络信息安全概念和特性（重点）

1.3.1 网络安全的特点

1. 机密性：指的是杜绝有用信息泄露给非授权的个人或者实体。
2. 完整性：信息保持非修改、非破坏、非丢失。
3. 可用性：可以被授权实体正常的访问。
4. 可控性：对网络信息传播和内容能够有效的控制。例如：直播的封禁、防火墙。^{非授权使用}
5. 不可否认性：所有参与者不可否认或者抵赖本人的真实身份。网络实名制。

1.4 行业篇

1.4.1 网络安全产品

1. 网络安全架构

从网络安全系统的角度：防火墙、入侵检测防护、虚拟专用网VPN、抗DDoS攻击（流量清洗）、网络行为审计、统一威胁管理系统UTM

老三样：防火墙、入侵检测、防病毒。

2. 防火墙：隔离区、内网

2. 密码算法与密码学

考试不会涉及到算法的具体细节与实现。

2.1 密码学的基本概念

加密：把数据和信息转换为不可识别的密文的过程。

解密：恢复数据和信息本来面目的过程。

以语言学为基础的密码方法失效。

现代密码方法主要在二进制字符串上进行。

密码方法所基于的问题只要被证明无法有效的解出，就是安全。

1. 基本术语：

明文：原有的信息

密文：经过加密变换后的内容

加密函数、解密函数

密钥：密码和明文之间的对应关系。

2. 分类一

根据算法和密钥是否分开来区分。

古典密码算法：依赖于算法本身的保密性。算法和密钥没有分开。

现代密码算法：算法和密钥是分开的，算法可以公开，但是密钥需要保密，依赖于密钥的保密性。可以大规模的应用，管理更加简单。

3. 分类二

根据加密和解密是否使用相同的密钥来区分。

对称密钥：使用相同的密钥，加密速度快，使用简单， n 个用户需要 $n(n-1)$ 个密钥 C_n^2 。

非对称密钥：使用不同的密钥。将加密密钥公开，解密密钥保密，又称为私钥。对于每一个用户，形成公私钥对。由此，不必保持信道的保密性， n 个用户需要 n 个密钥。但是算法速度比较慢。

混合加密体系。

4. 分类三

根据每次操作的数据单元是否分块。

分组密码：数据块之间的加密是独立的。操作一个数据块。

序列密码：数据块之间的加密是不独立的。加密解密操作于一个字节。

5. 分类四

根据明文加密后是否需要还原。

双向加密：加密后需要解密，目的是做信息的交换，保障数据的机密性。

单向加密：明文加密后不需要解密，目的是验证数据的完整性。

6. 密码分析

如果破解算法的代价大于被加密数据本身的价值，或者在信息的生命周期内无法破解，那么算法可能是安全的。

如果一个密码算法用可得到的资源不可以破解，那么算法是计算上安全的。

处理复杂性（计算量 CPU） 数据复杂性 存储复杂性

2.2 对称密钥密码算法

2.2.1 DES系列算法

分组加密算法，以64位为一个分组。密钥是56位。

对于每一个块，都要进行初始置换，末置换，以及16次的迭代，每次迭代。

64 51

• 每次迭代

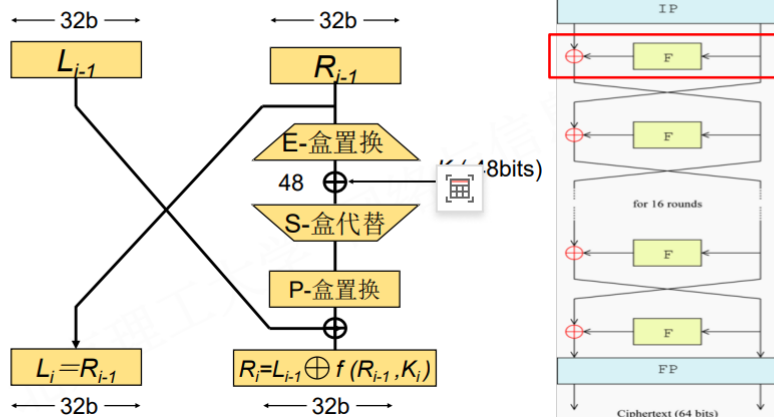


Figure 1

E-S-P

tripe DES:密钥的长度是112比特

(12

但是DES的密钥长度太短，在计算机算力指数增长下很容易被破解。

2.2.2 AES系列算法

AES区块长度固定为128bit，但是密钥的长度可以为128, 192, 256bit

对每一个块进行如下操作：

字节替换，行移位，列混淆，轮密钥加

秘钥分配通道

128

128 192 256

2.2.3 RC4算法

流 序列密码

产生一个随机伪bit流，与输入流的明文进行逐位的异或操作

解密时，用密文与伪随机bit流异或产生明文

异或操作具有对称性

$$\begin{cases} 0+0=0 \\ 1+1=0 \\ 1+0=1 \\ 0+1=1 \end{cases}$$

$$0 \times 1 \times 1 = 0; 0 \times 0 \times 0 = 0; 1 \times 1 \times 1 = 1; 1 \times 0 \times 0 = 1$$

Figure 2

2.3 非对称密钥密码算法

1. 对称密钥算法的缺点

密钥的分发需要经过安全通道、无法用于鉴别身份、密钥管理复杂 $O(N^2)$

$$O(n^2)$$

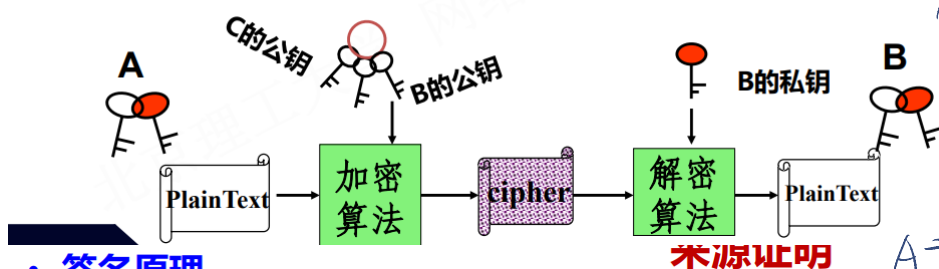
2. 非对称密钥密码系统

两个密钥，一公一私

公钥：用于加密和验证签名

私钥：用于解密和签名

— A向B发送消息，用B公钥加密；B收到后，用其私钥解密



• 签名原理

— A向B发送消息，用A的私钥加密——签名过程

— B收到密文后，用A的公钥解密——验证签名过程

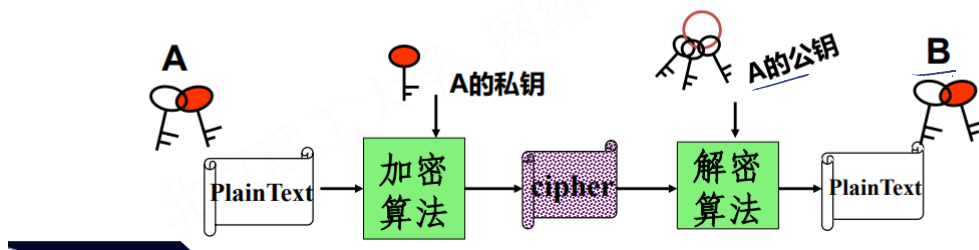


Figure 3

因为A的私钥别人是不知道的，所以能够得知信息的来源。

系列算法

对加密信息
非加密钥

A、B公钥

A → B

A → B

A 的公钥加密
B 用私钥

A → B 验证

A 的公钥

A 的私钥

加密

B

2.3.1 RSA算法

安全性基于分解极大数的困难性，数论基础 + 欧拉函数 + 欧拉定理

包括密钥生成算法加密算法和解密算法。

大数的因子分解是一个数论中的经典难题

2.3.2 ECC算法

解决椭圆曲线离散对数的困难性。

优势：用更短的密钥达到与RSA相同的安全性。

计算量小，处理速度快，存储空间和传输带宽占用少。

2.4 单向散列密码算法

1. 单向散列函数：hashing 哈希函数

使用hash函数，不管给多长的输入，都得到一个定长的输出，称之为摘要

从输入到摘要的计算容易，但是反推比较困难

输入不定长，输出定长

2.4.1 MD5 算法

对任意输入均产生128bit的摘要，简单紧凑

1. 填充，把输入长度变为 $N \times 512 + 448$
2. 将填充前的长度写入最后64位，总长度是512的整数倍
3. 初始化四个32位的寄存器
4. 处理每个报文分组，核心是4轮循环的压缩函数
5. 所有L个512bit的分组处理完成以后，第L阶段的输出便是128bit的报文摘要。

2.4.2 SHA算法

SHA2/SHA3已经成为新的标准

为什么淘汰MD5/SHA1

碰撞攻击定义：找到两个不同输入产生相同的哈希值。

能否在可接受的开销下完成碰撞攻击，是证明算法是否安全的基本逻辑。

存在严重安全影响，导致数字签名失效。

2.4.3 典型应用

HTTPS证书

资源完整性校验

数字证书

2.5 前沿密码技术

量子计算

2.6 总结

算法	类型	密钥/摘要长度	安全性	性能开销	典型应用场景
DES	对称加密（分组）	56位	不安全（暴力攻击）	低（硬件优化）	旧金融系统（已淘汰）
3DES	对称加密（分组）	168位（实际安全112位）	不太安全	高（三次加密）	旧支付终端系统升级（已淘汰）
AES	对称加密（分组）	128/192/256位	安全（抗量子攻击弱）	低（硬件加速优）	文件加密、TLS/SSL
RC4	对称加密（流）	40-2048位	不安全（密钥重用漏洞）	极低（流式处理）	旧版WiFi、WEP（已淘汰）
RSA	非对称加密	1024-4096位	安全	高（大数运算）	数字证书、密钥交换
ECC	非对称加密	160-521位	安全	中（比RSA高效）	移动设备、物联网

Figure 4

算法	类型	密钥/摘要长度	安全性	性能开销	典型应用场景
MD5	哈希函数	128位	不安全（严重碰撞漏洞）	极低	文件校验
SHA-1	哈希函数	160位	不安全（碰撞攻击）	低	旧数据验证（已淘汰）
SHA-2	哈希函数	224/256/384/512位	安全（SHA-256主流）	低	数据完整性、区块链
SHA-3	哈希函数	224/256/384/512位	安全性（独立结构）	中（海绵结构）	长期安全需求场景

Figure 5

3. 数据安全与隐私保护

3.1 数据安全概述

1. 数据安全威胁：
2. 数据的生命周期：
- 不同类型的数据，经历的生命周期也不同。典型的数据周期包括数据采集、存储、处理、传输、交换、销毁等阶段。

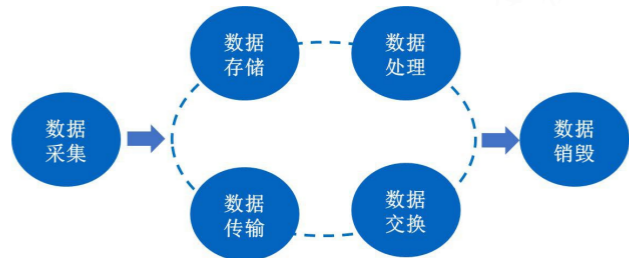


Figure 6

数据采集：保证采集源（证明采集点是经过安全验证的）和采集点数据（需要做匿名化和脱敏处理）

数据存储：存储在非安全的网络环境内，因此需要对其进行加密处理。

数据传输：数据层加密，网络层加密

数据处理：对动态数据进行一系列活动的组合，数据标记，清洗，数据模型设计等，访问控制是数据处理环节的安全措施。

数据交换：严格的身份认证，内置的安全性，匿名、脱敏、加密、水印。

数据销毁：彻底消失且无法通过任何手段恢复的过程。

3.2 数据安全中的密码应用

1. Linux登录密码安全

加密方法：crypt()早期的UNIX/Linux版本，基于DES

将用户的口令作为加密的密钥，加密一个64bit全部为0的数据块，然后将被加密的数据在一次加密，重复25次，最后用11个可打印的ASCII字符来表示

保存在/etc/passwd

破解方法：

没有算法可以恢复指令，唯一的方法是蛮力攻击。字典攻击。

增加干扰项：(salt)

12bit的随机数，与时间相关。

验证口令：

用户输入password

取salt

计算cipher2 = crypt(password,salt)

比较cipher2 and cipher1是不是相同。

crypt 是一个散列过程，UNIX采用MD5算法。

2. windows用户密码安全

windows是如何处理用户密码的？LAN Manger (LM) 哈希以及NT LAN Manger(NTLM)哈希。

所谓哈希操作，就是使用一个函数对任意长度的字符串进行一次数学运算，并且返回一个固定长度的字符串。

3. WIFI路由器密码安全

路由器的操作系统大多基于Linux内核，使用和Linux系统相同的密码管理方式，密码通常存在/etc/passwd和/etc/shadow中

python暴力破解

4. 密码应用开发

3.3 区块链原理及应用

账本! 账本公开, 资源消耗.

3.3.1 概念

账本

1. 数据角度: 一种几乎不能被更改的分布式数据库
2. 效果角度: 记录时间前后的、不可篡改的、可信任的数据库
3. 技术角度: 多种现有技术整合

3.3.2 原理

核心技术:

1. 哈希运算: 实现防篡改, 通过哈希构建默克尔树, 实现内容改变的快速检查
2. 数字签名: 每一个节点都有一份公私钥对, 接收到信息, 对数字签名进行验证, 完成消息完整性校验以及消息发送者身份合法性校验后, 触发后续处理。
3. P2P网络
4. 共识机制: 保障账本的一致性, 提高账本的容错性。
5. 智能合约: 计算机程序, 用于控制数字资产。

3.3.3 优点与不足

• 区块链的优点

- 分布式
- 去中心化
- 不可篡改, 撤销
- 开放性
- 自治性
- 匿名性



Figure 7

• 区块链的不足

- 交易账本必须公开
- 当数据越大伴随的性能问题
- 区块链的延迟性
- 能耗问题

Figure 8

3.3.4 分类

1. 根据网络范围: 公有链、私有链、联盟链

3.3.5 应用与开发

(应该不会考吧)

3.4 隐私保护与隐私计算

用户画像：基于用户线上行为数据抽象出他的信息全貌。

3.4.1 隐私保护的手段

本质：将隐私信息在适当的时间、以适当的方式分享给被授权的实体。

1. 隐私数据处理（脱敏）

泛化、抑制、解耦、置换、扰动。

泛化：将一类属性用一个更加宽泛的值代替。

准标识符, 属性

抑制：发布信息时将某个属性、属性的值一部分用*代替。

解耦和置换：都是去除准标识符和敏感属性之间的关联性，而不改变准标识符或敏感属性的值。解耦：将原始记录表分为两个表发布；置换：是把一个数据记录集合划分成组，在组内对敏感值进行置换，从而打乱标识符和敏感属性之间的对应关系。

扰动：用合成的数据值来替代原始的数据值。加噪，数据交换，合成数据生成。

2. K-匿名技术

通过概括，隐匿，发布精度较低的数据，使得同一个准标识符至少有K条记录，使观察者无法通过准标识符连接记录。

3.4.2 隐私计算

保证数据提供方不泄露原始数据的前提下，对数据进行分析计算的一系列信息技术，保证数据在流通与融合的过程中“可用但是不可见”。

方向：

1. 多方安全计算 密码学 不经意传输 比较课程分数 梨和苹果，只能看见自己的输入和属于自己的输出，保障不泄露中间数据。
2. 联邦学习 人工智能 从中央服务器下载现有的预测模型，使用本地数据训练，并上传到云。
3. 可信执行环境 基于硬件 机密性和完整性。

4. 访问控制和防火墙技术

4.1 访问控制原理

允许或者禁止某人使用某项资源的能力。

主体：提出资源访问请求

客体：被访问的资源实体

访问：对资源的使用

4.1.1 概念

→ 数据处理核心!

访问控制是信息安全保障机制的核心内容，是实现数据保密性和完整性机制的主要手段。

访问控制是^{为了}限制访问主体，对访问客体的访问权限，从而使计算机系统在合法的范围内使用。

作用：访问控制对机密性、完整性直接起的作用。

1. 描述

访问可以被描述为一个三元组 (s,o,a) subject 主题 object 客体 access 访问权限



Figure 9

ISO版本

访问控制流程描述 (ISO版本)

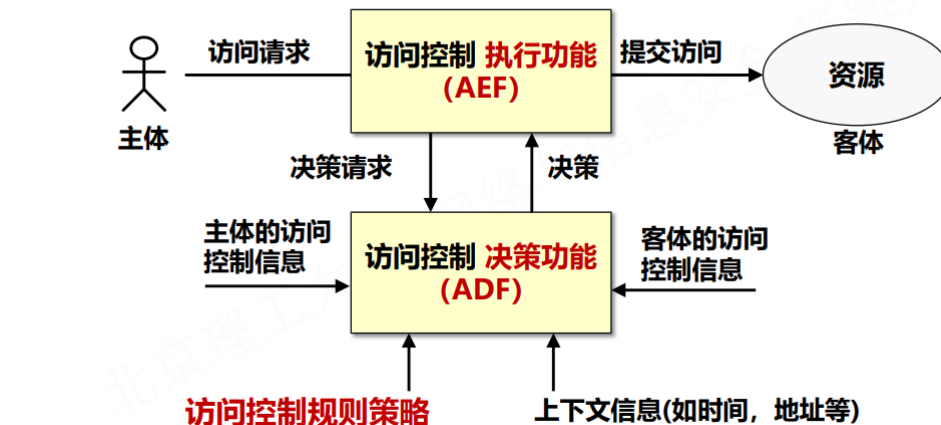


Figure 10

2. 访问控制模型

是实现严格访问控制策略所必须的。

自主访问控制模型 (DAC)、强制访问控制模型 (MAC)。

(1) 自主访问控制模型

每个客体有一个所有者，所有者按照其意愿把客体的访问控制权限授予其他主体。

控制灵活，是最为普遍的访问控制策略。

DAC的有效性，依赖于所有者对安全政策的正确理解和授予。

实现机制：访问控制表、访问控制矩阵。

ACM 矩阵

ACL 客定主

OL 主定客

– 访问控制矩阵 (ACM)：定义资源的用户权限

列：文件/进程——要访问的资源

	客体X	客体Y	客体Z	
主体A	读、修改、管理		读、修改、管理	行：用户/组/角色 ——请求的来源
主体B		读、修改、管理		
主体C	读	读、修改		
主体D	读	读、修改		

Figure 11

主体是要访问的人，客体是被访问的对象。

访问控制表ACL：客体决定主体

– 访问控制表 (Access Control List, ACL)：每个客体附加一个它可以访问的主体的明细表。客体决定主体

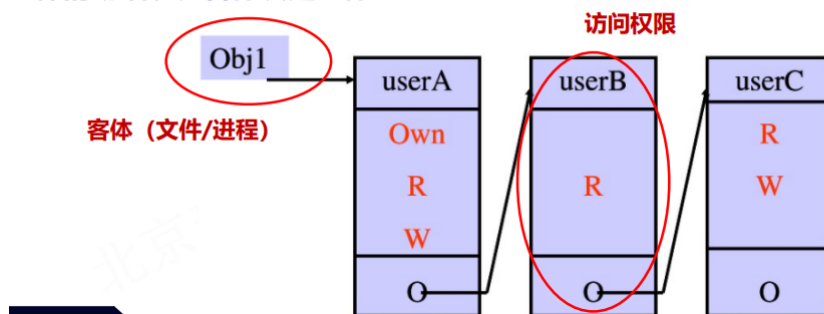


Figure 12

访问能力表 CL：主体决定客体

访问能力表 (Capacity List, CL)：每个主体附加一个该主体可访问的客体的明细表。主体决定客体

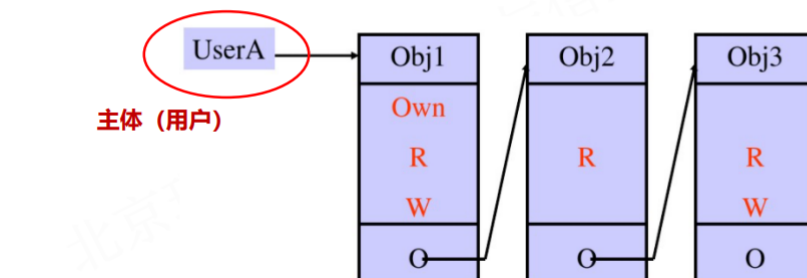


Figure 13

特点：根据主体的身份和授权来决定访问的模式

缺点：信息在移动过程中其访问权限关系会被改变。

(2) 强制访问控制 MAC

通过定义安全的级别，来达到访问控制

每个主体和客体分配一个固定的安全级别。

相比于DAC，访问控制粒度更细，安全性更高。

Windows强制完整性控制 MIC

(3) 基于角色的访问控制 (RBAC)

主客体数量和访问权限种类过多的解决办法。

设置多个角色并且每个角色都具备不同的权限。

用户组 $G = \{A, B, C, D\}$

角色：完成一项任务必须访问的资源及相应权限的集合 $R = \{(a1, o1), (a2, o2)\}$

授权管理：根据任务需要定义角色 为角色分配资源和操作权限。隔离了主体和权限。

(4) 基于属性的访问控制

组织规模持续扩大，导致RBAC角色爆炸

一种基于用户属性，资源属性，操作和环境属性等动态条件来做访问控制的模型。

考虑的是谁在什么条件下是否可以操作资源

更细粒度并且更加灵活。

所有的实体都会被描述统一为属性，所有的访问控制判定能够采取统一处理

分析与比较

· 四种访问控制模型

对比维	自主访问控制 ^{DAC}	强制访问控制 ^{MAC}	基于角色的访问控制 ^{RBAC}	基于属性的访问控制 ^{ABAC} ^{attribute}
典型场景	- 文件共享系统 - 小型组织	- 军事信息系统 - 机密数据处理	- 企业ERP系统 - 医院信息系统	- 云计算平台 - 物联网权限管理
应用实例	- Windows文件权限控制 - UNIX文件系统	- SELinux安全模块 - 多级安全数据库	- AWS IAM角色管理 - SAP权限系统	- AWS策略生成器 - XACML策略框架
策略驱动方式	用户自主决定	系统预设安全标签	管理员定义角色-权限关系	基于属性（用户/环境/资源）的策略规则
合规性优势	弱，易出现权限扩散	强，满足严格合规要求	较强，支持审计追踪	强，支持细粒度合规策略

Figure 14

4.1.2 访问控制的设计原则

1. 最小特权原则：每一个用户和进程都只拥有最小访问^权的集合，在最小保护域内执行。
2. 开放式设计原则：任何秘密不应该存在于访问控制的设计上（而应该在口令或者密钥当中），公开性设计
3. 特权的分隔原则：对客体的访问应该取决于不止一个条件被满足，即把访问某个客体的权利分解为多个子权力，由不同的主体掌握。例如主密钥的恢复必须多于 k 个子密钥，防止不良分子将主密钥邪路
4. 公共机制的最小化原则：公共机制是指多用户共享的机制，提供了隐蔽信道的可能，不利于系统安全，应该最小化。
5. 粒度、安全与复杂性：

细粒度访问控制：根据多种条件授予或撤销对关键系统和数据的访问的能力。例如，某个角色下的用户只有在公司工作至少一年后才能访问A。

粗粒度访问控制：在授予或者拒绝访问时只需要较低级别的特定性，例如特定角色下的任何用户都可以访问服务A。

过于细化的粒度可能会增加管理和维护的复杂性，过于粗化的粒度则可能无法满足具体的安全需求
避免陷入过度控制引发的管理熵增陷阱。

4.2 防火墙技术概述

4.2.1 什么是防火墙

内外通道

防火墙是在两个网络之间实施访问控制政策的一个或一组系统（硬件与软件的组合）

在两个新人程度不同的网络之间设置的，用于加强访问控制的软硬件保护设施。

两个安全域之间信息流的唯一通道。根据有关的安全正政策，控制进出网络的访问行为。

1. 分类

硬件防火墙：速度快，适合于局域网，指的是把防火墙程序做到芯片里

软件防火墙：速度较慢，适合单机，通过软件的方式来实现，但这类防火墙只能通过一定的规则来达到限制一些非法用户访问内部网的目的。

2. 作用：

防火墙可以在网络边界实施访问控制政策；

防火墙可以记录所有的访问；

防火墙可以隐藏内部网络，限制暴露内部用户点。

3. 不能做什么

不会自己配置，不能防止内部，无法控制没有经过防火墙的连接，无法防范全新的攻击，不能很好的实施内容防范，例如病毒。

4.3 防火墙核心技术

4.3.1 包过滤技术

防火墙能够区分包和限制包的能力叫做包过滤。工作在网络层。

基于以下信息对于每一个包进行检查：

IP源地址和目标地址、TCP/UDP源端口号和目标端口号、协议、包的大小、ICMP的消息类型。✓

依照访问控制列表（客体决定主体），指定目标数据包的属性特征以及对其的操作。

4.3.2 状态检测技术

追踪网络中活跃链接的状态，并根据此信息过滤网络包。

将属于一个连接的所有报文作为一个整体，工作在网络层和传输层。

是防火墙的基本功能，基本技术。

工作原理：会话表。涵盖一条链接撒谎给你两个方向上的信息。



Figure 15

4.3.3 应用层代理

工作在应用层，也叫做应用层网关。

具有隔断内网与外网通信的作用，保证内网不受到破坏。

支持身份认证的功能。

4.3.4 地址转换

类似路由器，工作在网络层，出了转发之外，还会完成地址转换。

不会提供额外的安全性，但是可以很好的隐蔽内部网络。

通过NAT网络被划分为内网和外网。

三种实现方式：静态转换、动态转换、端口多路复用。

4.3.5 下一代防火墙技术

skip，反正就是很高级就对了。。。

4.3.6 Web应用防火墙技术

集WEB防护、网页保护、负载均衡、应用交互于一体的WEB整体安全防护设备的一款产品。

维度	防火墙	WAF (Web Application Firewall)
防护层级	网络层、传输层 (L3-L4)	应用层 (L7, 专注于HTTP/HTTPS流量)
主要目标	控制网络流量 (IP/端口/协议)	防御Web应用攻击 (如SQL注入、XSS、CSRF)
防护对象	整个网络或子网	特定的Web应用或API接口
典型威胁	DDoS、端口扫描、未授权访问	OWASP Top 10漏洞 (如注入攻击、文件包含)

Figure 16

分类：分为硬件WAF，软件WAF，云WAF

工作原理：使用一组规则来区分正常请求和恶意请求

操作模型：

否定模型：基于黑名单，使用预设前面来阻止明显恶意的WEB流量

正面模型：仅允许根据特定配置标准的网络流量，但是可能会组织大量合法流量。

混合模型：混合上述两个模型。

工作模式：

透明代理模式：WAF在后台起到类似网桥的作用，对用户而言WAF是透明的。代理客户请求的作用。

反向代理模式：在客户端严重，WAF的地址就是真实服务器的地址，而实际上客户端的请求直接发送给WAF服务器，而不直接访问WEB服务器。

路由代理模式：与透明代理模式类似，但是是基于路由器转发客户端的请求

端口镜像模式：通过交换机镜像一份来自客户端的请求。

4.4 防火墙部署

DMZ非军事区，隔离区。

Mail服务器，Web服务器，数据库服务器等最容易受到攻击。

防火墙可以与病毒服务器 IDS 第三方认证的服务器联动。

4.5 防火墙配置与开发

4.5.1 Linux下的防火墙

iptables是与Linux内行人集成的IP信息包过滤系统，用于在Linux系统上控制IP信息包过滤和防火墙配置。

4.5.2 Windows下的防火墙

配置分为专用网络和公用网络

公用网络：当前的网络环境可能存在风险，咖啡厅，机场的公用网络。

专用网络：当前网络环境存在的风险较少，甚至没有。

4.6 总结

- 访问控制的三种模式，DAC、MAC、RBAC
- 防火墙概念
- 防火墙的核心技术（包过滤等）
- 防火墙的不同部署方式
- 防火墙与其它设备的联动
- 防火墙配置与开发

Figure 17

5. 系统安全技术

5.1 系统安全基础概述

1. 系统：信息系统，由一系列相互关联的组件构成，是通过信息的收集，处理、存储及发布为组织决策和组织控制提供支持的系统。
2. 系统安全：在系统生命周期内应用系统安全工程和系统安全管理方法，辨识系统种的隐患，并采取有效的控制措施，使其危险性最小，从而使系统在规定的性能、时间成本范围内达到最佳的安全程度。

两个方面：

运行方面：防火墙、网络行为审计、流量清洗

信息方面：加解密、防篡改、隐私保护

3. 目标：机密性、完整性、可用性、可追溯性

4. 保障业务连续性、保护敏感信息、提高市场竞争力。

5. 两个方面：抵御外部攻击，加强自身的健壮性。

5.2 操作系统安全性

5.2.1 概念

1. 操作系统是硬件与应用软件之间的桥梁，计算机资源的管理者，计算机软件的基础和核心。

2. 操作系统安全是操作系统自身及其所存储传输处理信息的保密性、完整性可用性。

5.2.2 安全威胁

1. 硬件安全威胁

侧信道攻击：通过加密软件或者硬件运行时产生的各种泄露信息获取密文信息。旁门左道的攻击方法都属于是侧信道攻击。

2. 软件安全威胁

逻辑炸弹，插入程序一段代码，当满足指定条件后引发恶意功能。

3. 数据安全威胁

机密性、完整性、数据合法性。

5.2.3 安全防护

1. 硬件安全机制

(1) 存储保护：最基本的要求，保护用户在存储器中的数据。保护单元越小，保护精度越高。防止用户程序对操作系统的影响，对进程的存储区域实行相互隔离。

(2) 运行保护

运行域是进程运行的区域。安全操作系统是分层设计，运行域也是基于保护环的等级式结构。在内层的环有最高特权，最大的环是最小特权。

(3) I/O保护

最复杂的保护，是只能由操作系统执行的一种特权操作。受到对设备的读写两种访问控制。

2. 软件安全机制

(1) 漏洞扫描及修复

提权攻击，攻击者通过利用系统中的漏洞或者配置错误，将自己的权限从较低提到较高级别。垂直提权：提升自己的权限等级，水平提权：扩大同一权限登记下的操作范围

5.3 数据库安全机制

5.3.1 安全威胁

数据库安全：防止数据库系统及其数据遭到泄露、篡改或破坏。

1. 应用层

(1) SQL注入：web应用程序对用户输入数据的合法性没有判断或过滤不严，攻击者在程序中事先定义的查询语句添加上额外的SQL语句，以此来欺骗数据库服务器，获取信息。

借用异常获取信息、借用页面响应时间判断是否注入。

2. DBMS层

权限提升漏洞，拒绝服务漏洞，缓冲区溢出。

3. 存储层

配置文件暴露密码

5.3.2 安全保障手段

在计算机系统中，安全措施是一级一级设置的。

• 数据库安全性控制的常用方法

计算机系统中，安全措施是一级一级层层设置的

- ①用户标识和鉴定 ----- ①合法用户准许进入系统
- ②存取控制 ----- ②用户只能执行合法操作
- ③视图 ----- ③操作系统有自己的保护措施
- ④审计 ----- ④数据以密文形式存储
- ⑤数据加密 ----- ④数据以密文形式存储

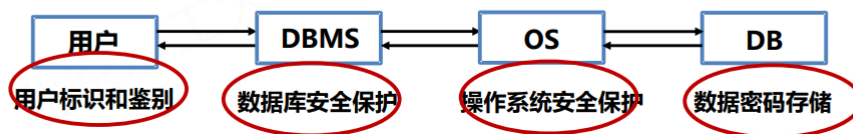


Figure 18

1. 视图

将要保密的数据对无权存取这些数据用户隐藏起来，对数据提供一定程度的安全保护。

2. 审计

审计日志，记录用户对数据库的所有操作。

日志

3. SQL注入的防御

普通用户与管理员之间权限要有严格的区分，/强迫使用参数化语句，通过参数来传递变量。

加强用户验证，测试字符串变量的内容，输入带有SQL语句的信息，返回输入不符合要求。

5.4 云系统的安全机制

5.4.1 简介

- 1. 云是通过互联网访问的服务器，以及在这些服务器上运行的软件和数据库。
- 2. 云计算是一种基于互联网的计算方式，通过这种方式，云的计算资源可以按照需要分配给各种终端设备。
- 3. 云系统是全套的计算设备
- 4. 部署方式：公有云、私有云、混合云。

5.4.2 云系统安全概念

- 1. 保护云计算环境，云中运行的程序以及保存的数据。
- 2. 不同：网络边界不可见、数据安全要求更高、责任主体更加复杂。

5.4.3 保障技术

- 1. 零信任安全：

组织不应信任边界内外的任何内容，必须在授予访问权限之前验证。最小权限，多次身份认证。

- 2. 全链路加密

数据在传输、计算、存储的过程叫做全链路。



Figure 19

5.5 病毒与防病毒技术

5.5.1 概述

- 1. 计算机病毒：破坏计算机功能或者毁坏数据，影响计算机使用，并且能自我复制的一组计算机指令或者程序代码。
- 2. 特点：
 - 传染性，搜寻其他符合其传播条件的程序或者介质，将自身代码插入其中。
 - 潜伏性：不会马上发作
 - 破坏性：
 - 隐藏性：不超过1MB，文件大小很小。
 - 可激发性：存在发作的设置条件。
 - 未经授权而执行：隐藏在正常程序中。
- 3. 区分概念

• 病毒 .exe

REPRESENT



- 在执行时设法将**自己复制到其他可执行代码中的恶意软件**
- 当被感染的可执行代码运行时，病毒也会同时被执行

• 蠕虫

- 能够**独立执行**并且可以将完整执行版本传播到**网络中其他主机**上的计算机程序

• 特洛伊木马

- 一种**看似完成有用功能的程序**，但具有隐蔽的、潜在的恶意功能，可以用来避开安全机制检查

Figure 20

病毒像“寄生虫”，依赖宿主文件传播

蠕虫像“传染病人”，独立且快速扩散

木马像“伪装间谍”，欺骗用户主动引入

4. 种类

宏病毒、脚本病毒、系统病毒、蠕虫病毒、木马病毒。

载荷系统破坏，数据损坏和勒索软件，物理损害。

5.5.2 计算机病毒原理

1. 感染机制

计算机病毒由一个载体传播到另一个载体，由一个系统进入另一个系统的过程。

2. 引导机制

3. 触发机制

4. 有效载荷

5.5.3 检测方法

1. 特征代码法：利用已知的特征代码来检测病毒，最简单
2. 校验和法：将正常文件的内容计算校验和，或者对其认证。
3. 行为检测：根据病毒特有的行为来检测病毒的方法。

5.5.4 防范方法

1. 杀毒引擎
2. 主动防御系统
3. 云计算

5.6 系统安全与审计技术

5.6.1 基本概念

1. 定义：收集和记录信息系统的各种日志、事件和流量信息，检查用户或者系统是否按照要求正常工作。
2. 合规性检查，核心特征：独立性 收集整理和分析数字证据。

3. 意义：保护机制，确定攻击事件和攻击源，证据。
4. 主机、网络安全、数据库、业务安全审计。

5.6.2 主要技术

数据处理，数据智能分析。

关联查询：找到多种数据一起来作为分析评价的证据。

关联分析：通过指定关联规则，对采集数据实时分析，规则匹配。

- 系统安全技术概述：运行、信息安全
- 操作系统安全机制：硬件、软件、数据安全、提权
- 数据库的安全机制：特殊查询构造
- 云系统的安全机制：零信任、五横三纵
- 病毒与防病毒技术：特征代码、校验和、行为检测
- 系统安全审计技术：关联查询、分析、挖掘、智能

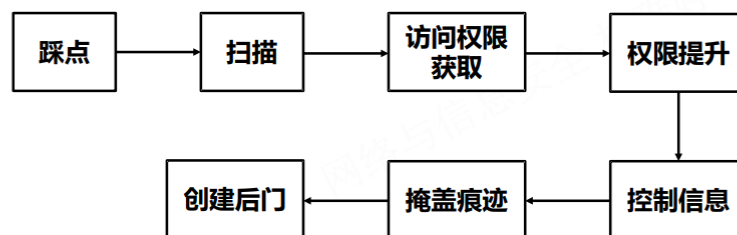
Figure 21

6. 网络攻防和入侵检测技术

6.1 网络攻防概述

1. 网络攻防：数据

• 网络攻击主要步骤



6.2 网络攻击技术

6.2.1 步骤

1. 隐藏IP：傀儡机、代理服务器
2. 踩点扫描：通过多种途径了解攻击目标的各方面情况。
3. 获得特权：由系统管理或者软件漏洞，获取系统的特权。
4. 种植后门：目的是保持对被入侵计算机长久的控制。
5. 隐身退出：不被管理员发现，清除登录日志和其他相关日志。

6.2.2 分类

✓ 1. 无访问权限攻击：社会工程学，钓鱼，分布式拒绝服务DDOS、ARP欺骗。

例如 DDOS、SYN Flood(TCP缺陷)、ICMP Smurf

2. 一般访问权限攻击：需要获取目标系统的一般用户访问权限才可以进行的攻击

口令绕过 $1 \text{ or } 1 = 1$ ，口令破解，SQL注入。

3. root访问权限攻击：需要获得目标系统root(管理员)权限，才可以进行的攻击。

后门木马，APT，Rootkit程序。

APT :advanced persistent threat. 高级持续威胁

6.2.3 防范工具

Goby

Burp suite

6.3 入侵检测技术 IDS

入侵检测系统在防火墙之后，防火墙是网络系统安全的第一道安全闸门。

6.3.1 概述

1. 定义：入侵，在非授权的情况下，试图存取信息，处理信息，或者破坏系统使系统不可靠、不可用。
2. 入侵防御：实时入侵检测系统。 IPS
3. 分类：基于主机的入侵检测系统、基于网络的入侵检测系统、混合型入侵检测系统

6.3.2 防御方法

- ✓
1. 误用检测：通过检测用户行为中与已知入侵行为模式类似的行为来检测系统中的入侵活动，是一种基于已有知识的检测。
 2. 异常检测：建立在“入侵行为与合法用户或者系统的正常或者期望的行为有偏差”，任何不符合以往活动规律的行为，都可以被视为入侵行为。误报率高，但是可以检测出未知的攻击。

6.4 网络防御系统部署

- NIDS（网络入侵检测系统）：IDS应当挂接在所有所关注流量都必经的链路上
- NIPS（网络入侵防御系统）：IPS直接串联到网络链路中

{ 主机——主防
网络——旁路防

7. 缓冲区溢出和漏洞利用

缓冲区溢出、社会工程学DDoS至今无解

攻击手段	攻击本质	为什么“无解”？
缓冲区溢出	技术漏洞 → 代码执行	永远可能有人写出漏洞代码
社会工程学	心理攻击 → 信息泄露	人性难改，反社会工程防御很难完全奏效
DDoS攻击	资源消耗 → 系统瘫痪	攻击门槛低、防守代价高

Figure 22

7.1 概述

1. 缓冲区：包含相同数据类型示例的一个连续的计算机内存块，或者程序运行期间在内存中分配的一个连续的区域
2. 内存：所填充的数据超出了原有的缓冲区边界。
3. 可能导致程序运行失败，系统崩溃，执行非授权指令，取得系统特权等后果

7.2 缓冲区溢出原理

7.2.1 程序在内存中的存放方式



Figure 23

代码区：存放代码，程序指令和只读数据

数据区：BSS段：未初始化全局或静态变量，数据段：初始化全局或静态变量

堆栈区：堆区：由程序员分配释放，栈区：由系统自动分配。

- 静态分配和动态分配的区别？ before / after
- static 声明的变量放在那里？ 静态
- malloc()函数分配的内存空间在哪里？
 - 堆空间 是 → 堆，系统 → 栈
- 频繁malloc和free对内存空间造成什么影响？
 - 内存碎片化、泄漏、性能下降、潜在漏洞

Figure 24

malloc 程序员自己分配的空间，在堆空间。

栈空间是由系统自动分配的。

7.2.2 溢出原理

如果在堆栈中压入的数据超过预先给堆栈分配的容量，就会发生堆栈溢出，从而导致程序运行失败，系统中所有缓冲区都有溢出的可能性。

7.2.3 种类

1. 栈溢出（最常用）

两个寄存器 SP(栈顶指针，随着数据入栈和出栈变化) BP（基址指针，标记栈中一个相对稳定的位置，函数的起点）

回答问题：

ESP:栈顶指针，^{随着}随着数据入栈和出栈而发生变化

EBP：基址指针，标记栈中一个相对稳定的位置，函数的起点

EIP：下一条指令的地址，压入栈顶，表示为RET，

ESP
← EIP
EBP

栈操作过程

- EIP（下一条指令的地址）压入栈顶，表示为RET
- 压入EBP
- 把当前栈指针ESP赋值给EBP
- 根据局部变量大小预留空间

Figure 25

防范：限制字符串的读写长度，限制在栈区域代码执行的权限。

2. 堆溢出

malloc free new delete

堆的操作是分配和回收

堆如何溢出？

比栈溢出难以发生

3. BSS溢出

buf1 buf2连续存贮，数组1的溢出内容会改变数组2 的值

7.2.4 32位系统与64位系统

处理数据能力不同

支持内存不同（寻址能力不同）

支持的处理器不同

软件兼容性

系统体积大小

7.3 缓冲区溢出的利用

7.3.1 步骤

1. 在程序的地址空间安排适当代码
2. 使控制流跳转到攻击代码

7.3.2 植入构造类型

NSR RNS AR

NSR: nops + shellcode + ret (返回地址)
RNS: ret + nops + shellcode
AR: 环境变量

7.3.3 shellcode

填充用的代码包括三部分：shellcode、返回地址和填充数据

7.3.4 shellcode

shellcode是完成特殊任务的自包含的二进制代码，一般用于获得系统特殊权限

对于不同的系统调用函数、不同的硬件平台，shellcode也会有区别

返回地址

即shellcode的入口地址，用于转移控制流

填充数据

辅助shellcode和返回地址，一般使用nop

7.3.5 危险函数

通常源于编程中选择了不安全的库函数。这些函数未对输出长度进行有效限制，导致数据覆盖相邻内存。

主要是没有限制输出输入长度，限制了就认为安全（？）

- 字符串操作函数：strcpy vs. strncpy, strcat vs. strncat, gets vs. fgets
- 内存操作函数：memcpy vs. memcpy_s, memset
- 输入输出函数：scanf, fscanf, vsprintf

Figure 26

7.4 漏洞利用与漏洞挖掘

7.4.1 漏洞的类别

按照可利用性指标以及影响性指标组的评分两部分，将漏洞分为高危漏洞、中危漏洞和低危漏洞。

7.4.2 漏洞发现

1. 人工分析：手工构造特殊输入条件
2. Fuzzing技术：实现原理是黑盒测试思想，使用大量的半有效数据作为输入
3. 补丁比对：通过比较补丁前后的二进制文件
4. 静态分析技术：不真正运行程序
5. 动态分析技术：执行过程中的状态信息来发现潜在问题

Fuzzing 模糊测试

6. 符号执行技术：用符号值代替真实值
7. 差分测试：同一个功能，在不同环境下的实现，比较差异
8. 供应链依赖：扫描第三方库
9. 漏洞模式匹配与AI驱动

7.4.3 0Day

T_0 — T_1 — T_2 —

泛指所有在官方发布该作品之前或者当天，由一些特别小组以一定的格式打包发布的数据内容。

0Day漏洞：被发现但是未被公开

1Day: 已被披露

NDay漏洞：发布官方补丁

8. 协议安全和VPN技术

8.1 互联网协议安全概述

网络层：IPV4在设计之初没有考虑安全性，IP欺骗与ICMP

传输层：TCP/UDP 没有考虑安全性

应用层：数百种应用型协议

根源是TCP/IP协议的缺陷

如何提高安全性？IPSec协议族 SSL/TLS协议组或DTLS.

- 方法一：数据加密？IPSec ESP
- 方法二：数据认证？IPSec AH 完整性
- 方法三：终端防护？传输模式
- 方法四：中间防护？隧道模式
- 方法五：深度握手？SSL/TLS 传统
- 方法六：专网专用？VPN

Figure 27

8.2 安全协议：IPSec

SSL/TLS

协议族/安全协议套件。是保护IP协议安全通信的标准，工作在第三层，网络层

SSL/TLS

8.2.1 概述

SSL/TLS

1. 包含的具体协议：

DTLS

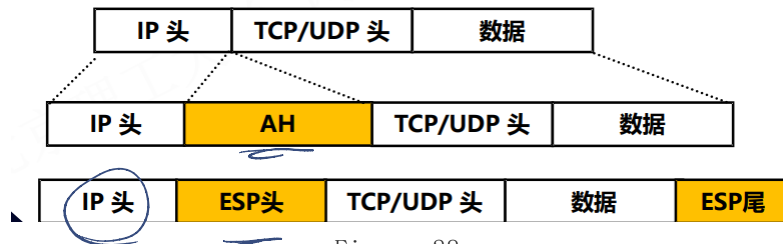
AH 认证头 对整个数据包进行认证，没有加密性，只完成完整性校验。 51

ESP 封装安全载荷：对有效载荷进行加密 50

8.2.2 工作模式

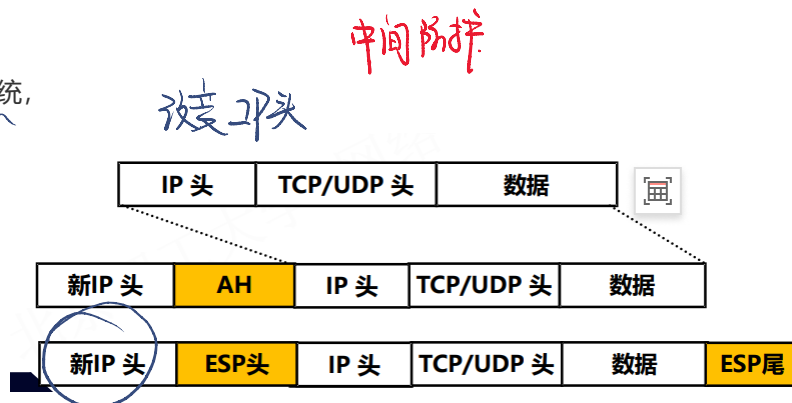
传输模式

只对IP数据包的有效负载进行加密或者认证，继续使用以前的IP头部，只对部分区域做修改。集成在操作系统当中。保证端对端通信



隧道模式

无需改变操作系统，



8.3 安全协议SSL(TLS)

8.3.1 概述

1. SSL 安全套接技术，机密性完整性
2. TLS 传输层安全

SSL协议由SSL记录协议、握手协议、密钥更改协议和告警协议组成。

基于TCP

8.4 DTLS

数据报传输层安全，提供了基于UDP传输场景下安全解决方案

DTLS协议由记录协议、握手协议、密钥更改协议和告警协议组成。

8.5 VPN技术

虚拟专用网络，在公网上建立一条加密隧道，实现远程安全访问私有网络的技术。

	VPN技术	专线技术
安全性	非常高 ，保护数据传输的完整性、保密性、不可抵赖性	比较高，建立在对电信部门相信的基础上
可扩展性	基于TCP/IP技术，只要网络可达，就可以方便扩展	当地运营商的支持，扩展很 不方便
投资成本	设备一次性投入，不需要支出每月的运营费用	需要每月支付 昂贵 的专线租用费用
移动支持	能对Internet上的内部移动用户安全接入，彻底消除地域差异	只能联通专线覆盖的网络
带宽	使用各种廉价的宽带介入方式，一般在1~100M	由于价格昂贵，一般租用的带宽都比较窄
升级	依赖于设备的升级	依赖于电信部门

Figure 30

关键技术：隧道技术 加解密技术 认证技术

种类：SSL/TLS VPN 或 DTLS VPN

IPSec VPN

Item	IPsec VPN	SSL VPN
需要安装、配置并管理客户端软件	需要	不需要
维护成本	高	低
防火墙需要改变和配置安全策略	很多	极少
可以进入所有内部网络	可以	不可以
任何计算机都可以使用	不可以	可以

Item	IPsec-VPN	SSL VPN
初始设备投资	相同	相同
客户端安装	费用高	不需要安装费用
大量远程用户推广	困难，费用高	简单方便
网络安全等级	好	更好
与应用系统整合	困难	简单
系统扩充性	具备	具备
用户端使用便利性	麻烦	简单

Figure 31

8.6 OpenSSL 编程

C语言作为开发语言

主要应用于创建证书、生成密钥、测试SSL/TLS链接

8.7 总结

- TCP/IP安全性的根源
- IPSec的两种模式和AH、ESP协议
- SSL协议所处层次和会话过程，DTLS及其区别
- VPN的概念
- IPSec VPN和SSL VPN的区别
- OpenSSL编程

Figure 32

9. 智能安全与数字媒体安全

9.1 智能系统安全

9.1.1 概论

1. 特征：模型赋能、数据驱动、用户交互。

2. 安全挑战：

模型安全：数据投毒、算法偏见（对于某些群体的不公平对待）

数据隐私安全：隐私泄露风险、人工智能系统易于成为攻击目标

应用部署安全：环境、底层框架漏洞，依赖库漏洞

相应的安全问题与特征——对应

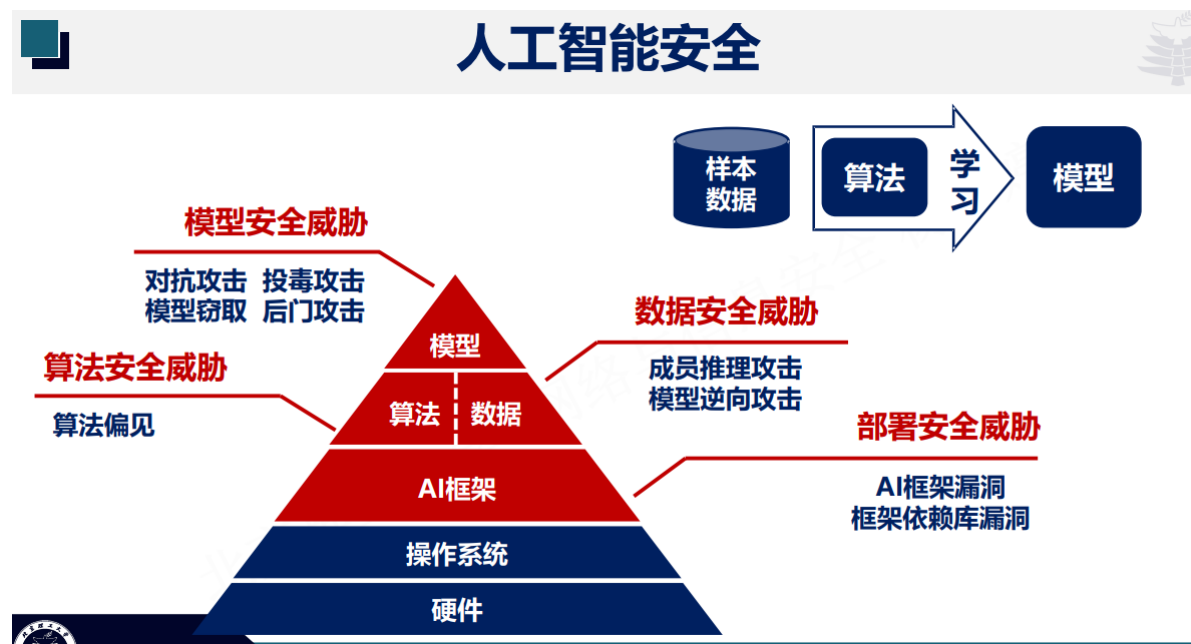


Figure 33

9.1.2 智能模型与算法安全

攻击类型

1. 对抗攻击

加入人类无法察觉的干扰/噪音 对抗样本 导致模型出错 原因在于模型高维线性的特性

又分为黑盒和白盒攻击

2. 逃逸攻击？对抗样本攻击中的一个重要例子，在不改变目标模型的情况下，通过构造特定的样本来欺骗目标系统的攻击。停止标志不能被检测到或者检测错误。

3. 投毒攻击

中毒样本 破坏模型的可用性或完整性 *模型倾斜, 反退化武器 后门攻击*

4. 模型窃取

复制一个功能相似甚至完全相同的AI模型，窃取模型本身的商业价值或者了解模型以对数据进行操作，提高对抗样本攻击的成功率。

防御策略

1. 模型完整性保障：

增加模型的鲁棒性，对抗训练（对抗样本），随机化（使得模型对噪声的容忍度增高）

模型完整性的保障：输入数据的预处理：去噪、利用现有的模型去生成数据。

算法本身会继承人类的偏见：输入数据偏见+算法规则偏见（平衡数据集、可解释性技术）

9.1.3 智能数据与数据保护

人工智能系统是数据驱动的。隐私安全

攻击类型

1. 成员推理攻击

判断某一个样本是否存在于模型的训练集中。

黑盒攻击：攻击者只能访问模型你，通过给定输入来观察输出。

原理：AI模型往往存在过拟合现象，在训练集和非训练集中表现存在差异，在训练数据上置信度更高。

如何训练攻击模型？构建一个与目标模型相似的影子模型。

2. 模型逆向攻击

通过模型的输出，反推训练集中的某条数据或者属性值

白盒攻击：攻击者知道模型的结构和参数。

知道人名，来推测人的图像。梯度下降法，迭代。

保护方式

1. 信息混淆

添加噪声，减少有效信息。混淆预测向量分布的差异，防御成员推理攻击。

2. 查询控制

黑盒攻击需要大量的查询，对查询行为进行特征提取，对攻击者限制或拒绝服务。

3. 隐私计算

利用差分隐私技术，对训练过程中的梯度信息进行一定程度的修饰。

3. 模型叠加

4. 法律法规

9.1.4 智能部署与防护手段

应用层：数据算法，训练得到模型

框架层：软件环境和工具集，简化开发过程。

框架依赖库：数学运算库numpy，数据处理库pandas

底层。

主要攻击AI框架和依赖库。

AI框架或者依赖库中有漏洞，就会影响到很多使用者。

终端模块漏洞

防护策略

1. 代码审查和安全测试（预防）模糊测试，注入随机不规则或者异常的数据
2. 入侵检测系统（检测）IDS 监控网络和系统活动，及时发现异常行为。网络入侵检测系统和主机入侵检测系统。
3. 漏洞监控与更新

9.2 数字媒体安全

基本安全特性：机密性、完整性、可用性、可控性、不可否认性。

附加安全特性：内容真实性、社会影响性、感官冗余性、产权归属性。

具体解释：

感官冗余性：多种数字媒体可以作为隐蔽通信的隐藏介质，声音、图片、文字可以通过修改顺序形成隐蔽的传输内容。例如LSB，利用生物对媒体内容的不敏感性，轻微改变颜色不会影响。

文本隐写

1. 文本媒体安全

文本隐写技术

修改式隐写：把文本中的某个词进行修改，修改冗余位来隐藏秘密信息。基于文本格式/文本内容。

生成式隐写：借助语言模型在自动生成文本载体

2. 音频安全

音频攻击

音频重放，语音合成，音频对抗（黑客通过微调音频的特定频率成分，人耳不宜观察，但是能欺骗语音系统），指令隐藏，超声波攻击，侧信道攻击（某些设备对声波敏感，特定频率的声音可以用来干扰或者操纵设备）。

防御措施

声纹检测、增强识别能力、对抗样本提高识别系统的鲁棒性、上下文理解能力与验证、限制智能设备的频率范围、对设备进行物理隔离或者声音过滤。

3. 图片安全之伪造检测技术

检测分类：

- 1. 主动取证，根据图片中的水印信息，后续检测是否发生篡改。
- 2. 被动取证：不依赖于预先嵌入的信息，检测图像是否由于篡改存在异常或者不一致性，基于图像内容（复制粘贴操作）。基于成像过程光线条件的取证方法

提取图像认证码：图像哈希。

4. 视频内容安全之内容审核安全

为什么？内容的合规性以及内容的版权。

(1) 异常内容检测

利用深度学习模型来进行异常内容的检测，高层次的图像理解操作。



Figure 34

(2) 视频拷贝检测

• 视频拷贝检测

- 全局拷贝检测：整个视频是否拷贝
- 局部拷贝检测：找出拷贝片段
- 典型的视频拷贝检测技术流程主要包含4个步骤：
 - 特征提取：对视频关键帧提取出相应特征向量
 - 建立索引：使用特征建立索引结构
 - 特征匹配：对视频特征在索引结构中匹配
 - 时间对齐：使用时间信息把拷贝帧整合成拷贝片段

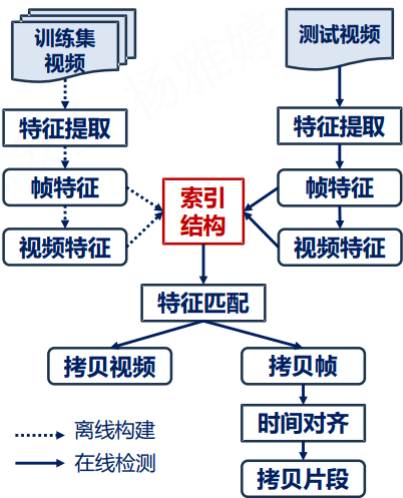


Figure 35

5. 游戏媒体安全之软件安全技术

账号财产安全

游戏环境安全：外挂，消极游戏

基础设施安全：服务器攻击，设备入侵，客户端（私服）